

Lab7 Report

Name: 时分

Student ID: PB23151799

Purpose

The purpose is to use a high-level programming language(e.g. C/C++) to implement the code in Lab1 ~ Lab4. Note that the algorithm needs to be consistent with what was used before. (e.g. Modulo operations cannot be replaced with %).

Here are program lists:

lab1: Unfold the Secret

lab2: Collatz Conjecture

lab3: Palindromic String

lab4: Sakiko's Savings

Principles

Lab1

A way to realize bitwise XOR A with B is:

1. $A + B = C$.
2. $(A \text{ AND } B) * 2 = \text{CARRY}$, CARRY is the carry bits in A+B.
3. $C - \text{CARRY} = A \text{ XOR } B$

The subroutine is as below:

```
int16_t lab1(int16_t n) {  
    // initialize  
    int16_t c, carry;  
    // calculation  
    c = n + STUDENT_ID_SECRET;  
    carry = (n & STUDENT_ID_SECRET) + (n & STUDENT_ID_SECRET);  
    // return value  
    return (c - carry);  
}
```

Lab2

There are some tasks to be solved:

1. To judge whether a_n is even or odd;
2. To realize $\frac{a_{n-1}}{2}$.

If a_n is even, the last bit of it is 0; if a_n is odd, the last bit is 1. So we can judge whether $a_n \text{ AND } 1 = 0$.

Division can be realized by doing subtraction and counting times it needs. To realize $a = a/2$ (a is an even number), we can repeat $a -= 2$ until $a = 0$, and record times we repeated.

```
int16_t lab2(int16_t n) {
    // initialize
    int16_t k = 0, a = n, count;
    // calculation
    while(a > 1){
        if((a & 1) == 0){
            //a is even
            count = 0;
            while(a > 0){
                a -= 2;
                count++;
            }
            a = count; //a = a/2
        }
        else{
            //a is odd
            a = a + a + a + 1;
        }
        k++;
    }
    // return value
    return k;
}
```

Lab3

Use 2 pointers in the process. At first, `ch1` points the start of the string, and `ch2` points the end of the string. `result` is 1 initially. In each loop, determine if `*ch1 == *ch2`. If not, the program sets `result` to 0 and stops looping; then `ch1` moves forwards, and `ch2` moves backwards. The loop ends when `ch1 >= ch2`. The code is as below:

```
int16_t lab3(char s1[], int n) {
    // initialize
    char *ch1 = s1, *ch2 = s1 + n - 1;
    int16_t result = 1;
    // calculation
    while(ch1 < ch2){
        if(*ch1 != *ch2){
            result = 0;
            break;
        }
        ch1++;
        ch2--;
    }
    // return value
    return result;
}
```

Lab4

$$S(n) = \begin{cases} 10, n = 0 \\ S(n-1) + Earn(n-1) - Spend(n-1), n > 0 \end{cases}$$

$$Earn(n) = \begin{cases} 6, n = 0 \\ Earn(n-1) \times 2, n > 0 \end{cases}$$

$$Spend(n) = \begin{cases} 2, n = 0 \\ 2, n > 0 \text{ and } Spend(n-1) \geq Earn(n-1) \\ Spend(n-1) \times 4, n > 0 \text{ and } Spend(n-1) < Earn(n-1) \end{cases}$$

Use 3 recursion subroutines to calculate. The code is as below:

```
int16_t lab4(int16_t n) {
    // initialize
    int16_t savings(int16_t n);
    int16_t result;
    // calculation
    result = savings(n);
    // return value
    return result;
}

int16_t earn(int16_t n){
    int16_t e;
    if(n == 0)
        return 6;
    else
    {
        e = earn(n-1);
        return (e+e);
    }
}

int16_t spend(int16_t n){
    int16_t s;
    if(n == 0)
        return 2;
    else if(spend(n-1) >= earn(n-1))
        return 2;
    else{
        s = spend(n-1);
        s = s + s;
        s = s + s;
        return s;
    }
}

int16_t savings(int16_t n){
    if(n == 0)
        return 10;
    else
        return savings(n-1) + earn(n-1) - spend(n-1);
}
```

Procedure

Bug1: The loop to realize $a = a/2$ in Lab2 can't end properly.

This is because the priority of `&` is lower than `==`. I fixed the bug by adding `()`.

Bug2: The data type of return values doesn't match the data type of the functions.

The data type of the functions are `int16_t`, so the return values should be `int16_t`.

Results

1. The input (test.txt) is:

```
194
6
abdcba 7
5
```

The output on screen is:

```
===== 1ab1 =====
189
===== 1ab2 =====
8
===== 1ab3 =====
1
===== 1ab4 =====
144
```

2. The input (test.txt) is:

```
100
56
abdcba 6
9
```

The output on screen is:

```
===== 1ab1 =====
27
===== 1ab2 =====
19
===== 1ab3 =====
0
===== 1ab4 =====
304
```

All results are correct.